SPRING DOCS

Microservices with Spring

Service Discovery, API Gateway, Resilience & Distributed Patterns

Java Agentic AI — Knowledge Base Reference

1. Monolith vs Microservices

Aspect	Monolith	Microservices
Deployment	Single deployable unit	Independently deployable services
Scaling	Scale the whole app together	Scale individual services based on their load
Technology	Usually one stack throughout	Each service can use its own stack/language
Data	Single shared database	Database-per-service (no shared schema)
Failure isolation	One bug can bring down the whole app	A failing service can be isolated/degraded gracefully
Complexity	Simpler to develop/test/deploy initially	Higher operational complexity — network calls, distributed data, obser

> ***Note:*** *Microservices trade development simplicity for independent scalability, deployability, and team autonomy — they solve organizational and scaling problems, not correctness problems, and introduce real distributed-systems complexity (network failures, eventual consistency, observability).*

2. Core Building Blocks (Spring Cloud)

Concern	Spring Cloud component
Service discovery	Eureka (Netflix OSS) / Consul
API Gateway	Spring Cloud Gateway
Client-side load balancing	Spring Cloud LoadBalancer (replaced Netflix Ribbon)
Centralized configuration	Spring Cloud Config Server
Resilience (circuit breaking, retries)	Resilience4j (replaced Netflix Hystrix)
Distributed tracing	Spring Cloud Sleuth + Zipkin / Micrometer Tracing
Inter-service messaging	Spring Cloud Stream (Kafka/RabbitMQ binder)

3. Service Discovery

In a dynamic environment where service instances scale up/down and get new IPs, hardcoding hostnames is impractical. A service registry (like Eureka) lets services register themselves on startup and lets clients discover healthy instances by logical name.

```
// Eureka Server
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServerApp { ... }

// Eureka Client (any microservice)
@SpringBootApplication
@EnableDiscoveryClient
public class OrderServiceApp { ... }
```

```
// application.yml
eureka:
client:
    service-url:
     defaultZone: http://localhost:8761/eureka

// Calling another service by logical name via a load-balanced RestTemplate/WebClient
@LoadBalanced
@Bean
public RestTemplate restTemplate() { return new RestTemplate(); }

restTemplate.getForObject("http://inventory-service/api/stock/{id}", Stock.class,
productId);
```

4. API Gateway

A single entry point that routes external client requests to the appropriate internal service, centralizing cross-cutting concerns: authentication, rate limiting, request/response logging, and routing — clients never need to know internal service topology.

```
spring:
cloud:
    gateway:
     routes:
        - id: order-service
          uri: lb://ORDER-SERVICE
          predicates:
             - Path=/api/orders/**
          filters:
             - StripPrefix=1
        - id: user-service
          uri: lb://USER-SERVICE
          predicates:
             - Path=/api/users/**
```

5. Inter-Service Communication

Style	Examples	Trade-off
Synchronous (request/response)	REST over HTTP, gRPC	Simple mental model, but creates tight temporal coupling — caller b
Asynchronous (messaging)	Kafka, RabbitMQ, Spring Cloud Stream	Loose coupling, resilience to temporary outages, but eventual consis

```
// Declarative HTTP client - Spring Cloud OpenFeign
@FeignClient(name = "inventory-service")
public interface InventoryClient {
    @GetMapping("/api/stock/{productId}")
    StockDto getStock(@PathVariable Long productId);
}

@Service
public class OrderService {
```

```
    private final InventoryClient inventoryClient;
    public void placeOrder(Order order) {
        StockDto stock = inventoryClient.getStock(order.getProductId());
        // ...
    }
}
```

6. Resilience Patterns

6.1 Circuit Breaker

Prevents cascading failures: if a downstream service repeatedly fails, the circuit 'opens' and further calls fail fast (or fall back) instead of waiting on a doomed dependency, giving it time to recover.

```
@CircuitBreaker(name = "inventoryService", fallbackMethod = "fallbackStock")
public StockDto getStock(Long productId) {
    return inventoryClient.getStock(productId);
}

public StockDto fallbackStock(Long productId, Throwable t) {
    return new StockDto(productId, 0, "Service temporarily unavailable");
}
```

Circuit state	Behavior
CLOSED	Requests flow normally; failures are counted
OPEN	Failure threshold exceeded; requests fail fast without calling downstream
HALF_OPEN	After a wait duration, a limited number of trial requests test if the dependency recovered

6.2 Retry, Rate Limiter, Bulkhead

```
@Retry(name = "inventoryService", fallbackMethod = "fallbackStock")
@RateLimiter(name = "inventoryService")
@Bulkhead(name = "inventoryService")
public StockDto getStock(Long productId) { ... }
```

- **Retry** — automatically re-attempts a failed call a bounded number of times (ideally with backoff), useful for transient failures.
- **Rate Limiter** — caps the number of calls per time window, protecting both caller and callee from overload.
- **Bulkhead** — isolates resource pools (thread pools/semaphores) per dependency so one slow downstream service can't exhaust resources needed by calls to other services.

7. Data Management: Database per Service & Saga Pattern

Each microservice owns its own database, preventing tight coupling through shared schemas. This makes multi-service transactions impossible with a traditional ACID transaction, so the **Saga pattern** coordinates a sequence of local transactions, each publishing an event that triggers the next step, with

compensating actions to undo prior steps if a later one fails.

```
// Choreography-based saga (event-driven, no central coordinator)
OrderService: creates order (PENDING) -> publishes OrderCreatedEvent
InventoryService: reserves stock -> publishes StockReservedEvent (or
StockReservationFailedEvent)
PaymentService: charges customer -> publishes PaymentCompletedEvent (or PaymentFailedEvent)
OrderService: on PaymentFailedEvent -> compensating action: cancel order, release stock
```

8. Centralized Configuration

```
// Config Server
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApp { ... }

// Client application.yml
spring:
config:
    import: "optional:configserver:http://localhost:8888"
```

9. Observability

- **Centralized logging** — aggregate logs from all instances (ELK stack, Loki) since logs are scattered across many independently-scaled processes.
- **Distributed tracing** — a trace ID propagates across service boundaries so a single logical request can be followed through every hop (Micrometer Tracing + Zipkin/Jaeger).
- **Metrics** — Actuator + Micrometer expose per-service metrics scraped by Prometheus and visualized in Grafana.

10. Quick Interview Recap

- Why database-per-service? Prevents tight coupling via shared schema, letting each service evolve and scale its data model independently — at the cost of needing patterns like Saga for cross-service consistency.
- What does a circuit breaker actually do? Stops calling a repeatedly-failing dependency for a cooldown period, failing fast (or using a fallback) instead of piling up blocked threads/timeouts.
- Choreography vs orchestration saga? Choreography: services react to each other's events with no central coordinator. Orchestration: a central saga orchestrator explicitly tells each service what to do next.
- Why is service discovery needed? Instances scale dynamically and get new network locations; a registry lets services find each other by logical name rather than hardcoded addresses.